

Classifiez automatiquement des informations

Projet 4 – OpenClassrooms Data Scientist Machine Learning

Elodie Chen

A large blue triangle is positioned in the bottom right corner of the slide, pointing towards the top right.

Description des données

Description des données à disposition

Trois fichiers CSV provenant de sources différentes sont fournis :

1. Système SIRH

- Informations sur les employés : fonction, âge, salaire, ancienneté, etc.
- Informations sociodémographiques.

2. Évaluations de performance

- Notes annuelles de performance des employés.
- Notes de satisfaction données par les employés.

3. Sondage sur le bien-être des employés

- Rempli annuellement par tous les employés.
- Aide à identifier les leviers pour améliorer le bien-être.
- Contient un témoin indiquant si l'employé a quitté l'entreprise ou non.

```

1 import pandas as pd
2
3 df_eval = pd.read_csv('extrait_eval.csv')
4 df_sirh = pd.read_csv('extrait_sirh.csv')
5 df_sondage = pd.read_csv('extrait_sondage.csv')
6
7 df_eval

```

✓ 0.0s

Python

	satisfaction_employee_environnement	note_evaluation_precedente	niveau_hierarchique_poste	satisfaction_employee_nature_travail	satisfaction_employee_equipe	satisfaction_employee_equilibre_pro_perso	eval_number	note_evaluation
0	2	3	2	4	1	1	E_1	
1	3	2	2	2	4	3	E_2	
2	4	2	1	3	2	3	E_4	
3	4	3	1	3	3	3	E_5	
4	1	3	1	2	4	3	E_7	
...	...	...	...	...	...	...	...	...
1465	3	4	2	4	3	3	E_2061	
1466	4	2	3	1	1	3	E_2062	
1467	2	4	2	2	2	3	E_2064	
1468	4	2	2	2	4	2	E_2065	
1469	2	4	2	3	1	4	E_2068	

1470 rows x 10 columns

1 df_sirh

✓ 0.0s

Pyt

	id_employee	age	genre	revenu_mensuel	statut_marital	departement	poste	nombre_experiences_precedentes	nombre_heures_travailless	annee_experience_totale	annees_dans_l_entreprise	annees_dans_le_poste_actuel
0	1	41	F	5993	Célibataire	Commercial	Cadre Commercial	8	80	8	6	
1	2	49	M	5130	Marié(e)	Consulting	Assistant de Direction	1	80	10	10	
2	4	37	M	2090	Célibataire	Consulting	Consultant	6	80	7	0	
3	5	33	F	2909	Marié(e)	Consulting	Assistant de Direction	1	80	8	8	
4	7	27	M	3468	Marié(e)	Consulting	Consultant	9	80	6	2	
...	...	...	...	...	...	...	...	...	...	...	...	...
1465	2061	36	M	2571	Marié(e)	Consulting	Consultant	4	80	17	5	
1466	2062	39	M	9991	Marié(e)	Consulting	Manager	4	80	9	7	
1467	2084	27	M	6142	Marié(e)	Consulting	Tech Lead	1	80	6	6	
1468	2065	49	M	5390	Marié(e)	Commercial	Cadre Commercial	2	80	17	9	
1469	2068	34	M	4404	Marié(e)	Consulting	Consultant	2	80	6	4	

1470 rows x 12 columns

1 df_sondage

✓ 0.0s

Python

	a_quitte_l_entreprise	nombre_participation_pee	nb_formations_suivies	nombre_employe_sous_responsabilite	code_sondage	distance_domicile_travail	niveau_education	domaine_etude	ayant_enfants	frequence_deplacement	annee
0	Oui	0	0	1	1	1	2	Infra & Cloud	Y	Occasionnel	
1	Non	1	3	1	2	8	1	Infra & Cloud	Y	Frequent	
2	Oui	0	3	1	4	2	2	Autre	Y	Occasionnel	
3	Non	0	3	1	5	3	4	Infra & Cloud	Y	Frequent	
4	Non	1	3	1	7	2	1	Transformation Digitale	Y	Occasionnel	
...	...	...	...	...	...	...	...	...	...	...	...
1465	Non	1	3	1	2061	23	2	Transformation Digitale	Y	Frequent	
1466	Non	1	5	1	2062	6	1	Transformation Digitale	Y	Occasionnel	
1467	Non	1	0	1	2064	4	3	Infra & Cloud	Y	Occasionnel	
1468	Non	0	3	1	2065	2	3	Transformation Digitale	Y	Frequent	
1469	Non	0	3	1	2068	8	3	Transformation Digitale	Y	Occasionnel	

1470 rows x 12 columns

```

1 import pandas as pd
2
3 df_eval = pd.read_csv('extract_eval.csv')
4 df_sirh = pd.read_csv('extract_sirh.csv')
5 df_sondage = pd.read_csv('extract_sondage.csv')
6
7 df_eval

```

	satisfaction_employee_environnement	note_evaluation_precedente	niveau_hierarchique_poste	satisfaction_employee_nature_travail	satisfaction_employee_equipe	satisfaction_employee_equilibre_pro_perso	eval_number	note_eval
0	2	3	2	4	1	1	E_1	
1	3	2	3	2	4	3	E_2	
2	4	2	1	3	2	3	E_4	
3	4	3	1	3	3	3	E_5	
4	1	3	1	2	4	3	E_7	
...	...	...	...	...	...	...	...	...
1465	3	4	2	4	3	3	E_2061	
1466	4	2	3	1	1	3	E_2062	
1467	2	4	2	2	2	3	E_2064	
1468	4	2	2	2	4	2	E_2065	
1469	2	4	2	3	1	4	E_2068	

1470 rows x 9 columns

```

1 df_sirh

```

	id_employee	age	genre	revenu_mensuel	statut_marital	departement	poste	nombre_experiences_precedentes	nombre_heures_travailless	annee_experience_totale	annees_dans_l_entreprise
0	1	41	F	5993	Célibataire	Commercial	Cadre Commercial	8	80	8	
1	2	49	M	5130	Marié(e)	Consulting	Assistant de Direction	1	80	10	
2	4	37	M	2090	Célibataire	Consulting	Consultant	6	80	7	
3	5	33	F	2909	Marié(e)	Consulting	Assistant de Direction	1	80	8	
4	7	27	M	3468	Marié(e)	Consulting	Consultant	9	80	6	
...	...	...	...	...	...	...	...	...	...	...	...
1465	2061	36	M	2571	Marié(e)	Consulting	Consultant	4	80	17	
1466	2062	39	M	9991	Marié(e)	Consulting	Manager	4	80	9	
1467	2064	27	M	6142	Marié(e)	Consulting	Tech Lead	1	80	6	
1468	2065	49	M	5390	Marié(e)	Commercial	Cadre Commercial	2	80	17	
1469	2068	34	M	4404	Marié(e)	Consulting	Consultant	2	80	6	

1470 rows x 12 columns

```

1 df_eval.info()

```

0.0s

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 10 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   satisfaction_employee_environnement      1470 non-null   int64
1   note_evaluation_precedente               1470 non-null   int64
2   niveau_hierarchique_poste               1470 non-null   int64
3   satisfaction_employee_nature_travail     1470 non-null   int64
4   satisfaction_employee_equipe             1470 non-null   int64
5   satisfaction_employee_equilibre_pro_perso 1470 non-null   int64
6   eval_number                             1470 non-null   object
7   note_evaluation_actuelle                 1470 non-null   int64
8   heure_supplementaires                   1470 non-null   object
9   augmentation_salaire_precedente         1470 non-null   object
dtypes: int64(7), object(3)
memory usage: 115.0+ KB

```

```

1 df_sirh.info()

```

0.0s

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 12 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   id_employee                             1470 non-null   int64
1   age                                     1470 non-null   int64
2   genre                                   1470 non-null   object
3   revenu_mensuel                         1470 non-null   int64
4   statut_marital                         1470 non-null   object
5   departement                           1470 non-null   object
6   poste                                  1470 non-null   object
7   nombre_experiences_precedentes         1470 non-null   int64
8   nombre_heures_travailless              1470 non-null   int64
9   annee_experience_totale                 1470 non-null   int64
10  annees_dans_l_entreprise                1470 non-null   int64
11  annees_dans_le_poste_actuel             1470 non-null   int64
dtypes: int64(8), object(4)
memory usage: 137.9+ KB

```

```
1 df_sondage
```

✓ 0.0s

	a_quitte_l_entreprise	nombre_participation_pee	nb_formations_suivies	nombre_employee_sous_responsabilite	code_sondage	distance_domicile_travail	niveau_education	
0	Oui	0	0	1	1	1	1	2
1	Non	1	3	1	2	8	1	1
2	Oui	0	3	1	4	2	2	2
3	Non	0	3	1	5	3	4	4
4	Non	1	3	1	7	2	1	1
...	...	...	...	...	...	...	...	...
1465	Non	1	3	1	2061	23	2	2
1466	Non	1	5	1	2062	6	1	1
1467	Non	1	0	1	2064	4	3	3
1468	Non	0	3	1	2065	2	3	3
1469	Non	0	3	1	2068	8	3	3

1470 rows x 12 columns

```
1 df_sondage.info()
```

✓ 0.0s

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1470 entries, 0 to 1469

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	a_quitte_l_entreprise	1470 non-null	object
1	nombre_participation_pee	1470 non-null	int64
2	nb_formations_suivies	1470 non-null	int64
3	nombre_employee_sous_responsabilite	1470 non-null	int64
4	code_sondage	1470 non-null	int64
5	distance_domicile_travail	1470 non-null	int64
6	niveau_education	1470 non-null	int64
7	domaine_etude	1470 non-null	object
8	ayant_enfants	1470 non-null	object
9	frequence_deplacement	1470 non-null	object
10	annees_depuis_la_derniere_promotion	1470 non-null	int64
11	annes_sous_responsable_actuel	1470 non-null	int64

dtypes: int64(8), object(4)

memory usage: 137.9+ KB

1470 lignes, 32 colonnes

Fusion des données

```
df = pd.merge(df_eval, df_sirh, how='inner', on='id_employe')
```

```
1 df_sondage['code_sondage'] = df_sondage['code_sondage'].astype('str')
2
3 (df_sondage['code_sondage'] == df['id_employe']).value_counts()
```

```
True    1470
Name: count, dtype: int64
```

```
1 df = pd.merge(df, df_sondage, left_on='id_employe', right_on='code_sondage', how='inner')
2 df = df.drop(columns='code_sondage')
```

```
1 df.info()
```

```
✓ 0.0s
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1470 entries, 0 to 1469
```

```
Data columns (total 32 columns):
```

#	Column	Non-Null Count	Dtype
0	id_employe	1470 non-null	object
1	satisfaction_employe_environnement	1470 non-null	int64
2	note_evaluation_precedente	1470 non-null	int64
3	niveau_hierarchique_poste	1470 non-null	int64
4	satisfaction_employe_nature_travail	1470 non-null	int64
5	satisfaction_employe_equipe	1470 non-null	int64
6	satisfaction_employe_equilibre_pro_perso	1470 non-null	int64
7	note_evaluation_actuelle	1470 non-null	int64
8	heure_supplementaires	1470 non-null	object
9	augmentation_salaire_precedente	1470 non-null	object
10	age	1470 non-null	int64
11	genre	1470 non-null	object
12	revenu_mensuel	1470 non-null	int64
13	statut_marital	1470 non-null	object
14	departement	1470 non-null	object
15	poste	1470 non-null	object
16	nombre_experiences_precedentes	1470 non-null	int64
17	nombre_heures_travailless	1470 non-null	int64
18	annee_experience_totale	1470 non-null	int64
19	annees_dans_l_entreprise	1470 non-null	int64
20	annees_dans_le_poste_actuel	1470 non-null	int64
21	a_quitte_l_entreprise	1470 non-null	object
22	nombre_participation_pee	1470 non-null	int64
23	nb_formations_suivies	1470 non-null	int64
24	nombre_employe_sous_responsabilite	1470 non-null	int64
25	distance_domicile_travail	1470 non-null	int64
26	niveau_education	1470 non-null	int64
27	domaine_etude	1470 non-null	object
28	ayant_enfants	1470 non-null	object
29	frequence_deplacement	1470 non-null	object
30	annees_depuis_la_derniere_promotion	1470 non-null	int64
31	annees_sous_responsable_actuel	1470 non-null	int64

```
dtypes: int64(21), object(11)
```

```
memory usage: 367.6+ KB
```


EDA

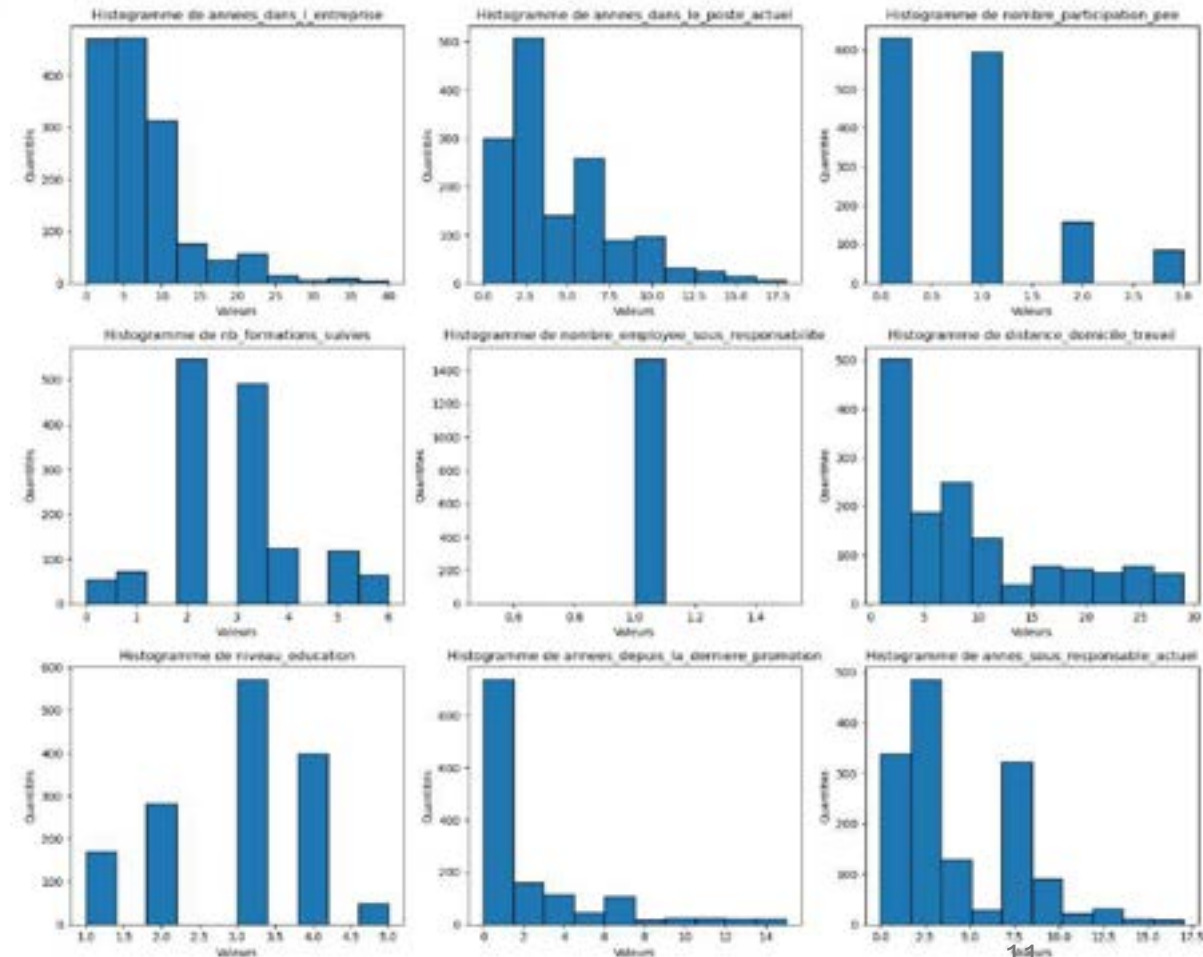
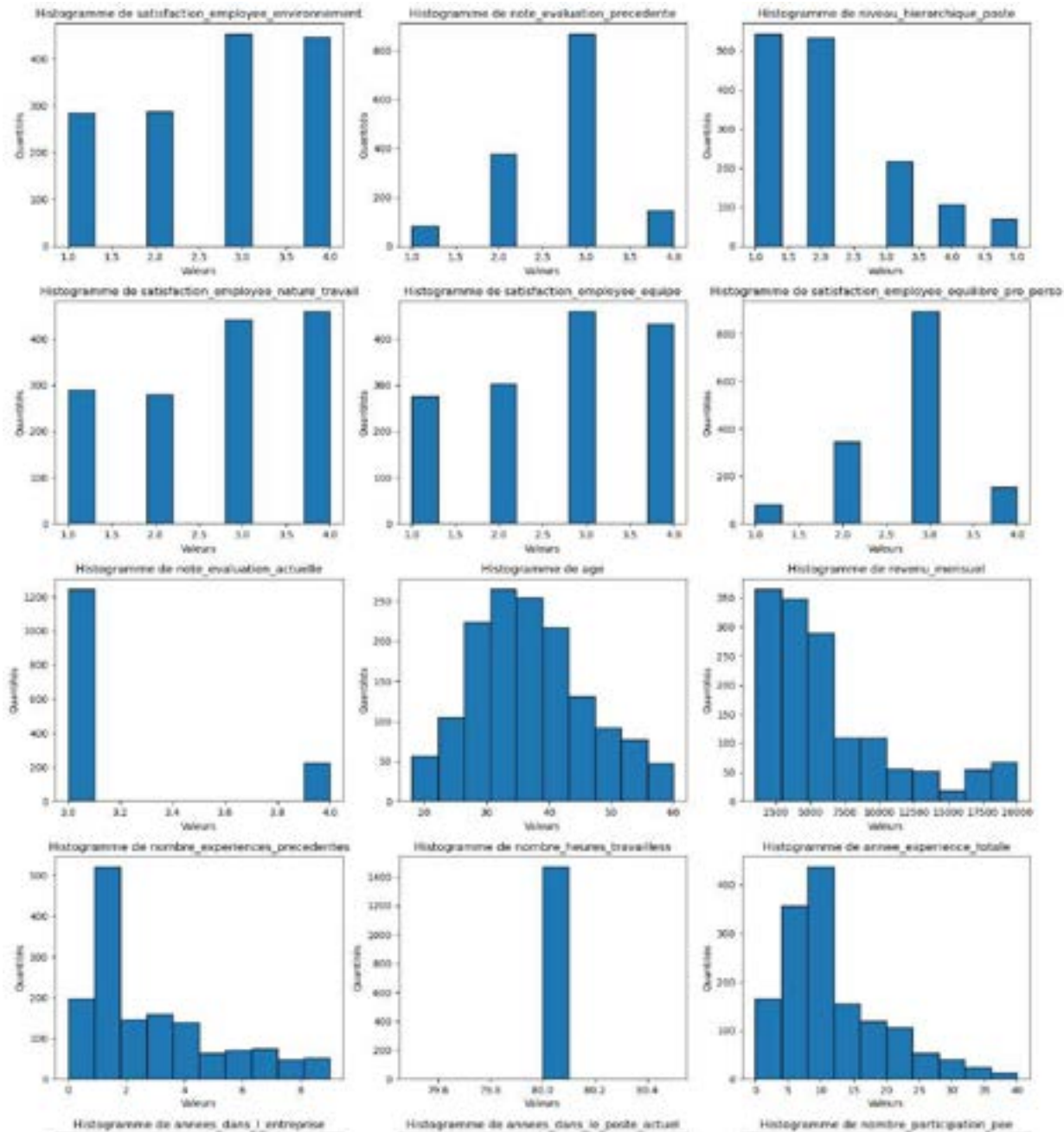
```
1 df.describe()
```

✓ 0.0s

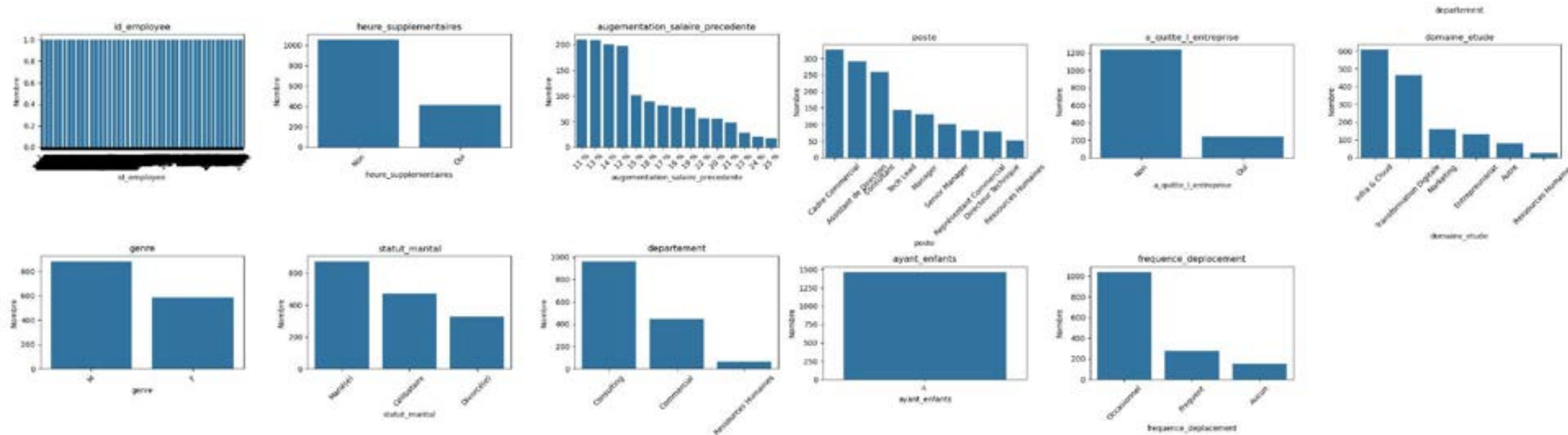
	satisfaction_employee_environnement	note_evaluation_precedente	niveau_hierarchique_poste	satisfaction_employee_nature_travail
count	1470.000000	1470.000000	1470.000000	1470.000000
mean	2.721769	2.729932	2.063946	2.728571
std	1.093082	0.711561	1.106940	1.102846
min	1.000000	1.000000	1.000000	1.000000
25%	2.000000	2.000000	1.000000	2.000000
50%	3.000000	3.000000	2.000000	3.000000
75%	4.000000	3.000000	3.000000	4.000000
max	4.000000	4.000000	5.000000	4.000000

satisfaction_employee_equipe	satisfaction_employee_equilibre_pro_perso	note_evaluation_actuelle	age	revenu_mensuel
1470.000000	1470.000000	1470.000000	1470.000000	1470.000000
2.712245	2.761224	3.153741	36.923810	6502.931293
1.081209	0.706476	0.360824	9.135373	4707.956783
1.000000	1.000000	3.000000	18.000000	1009.000000
2.000000	2.000000	3.000000	30.000000	2911.000000
3.000000	3.000000	3.000000	36.000000	4919.000000
4.000000	3.000000	3.000000	43.000000	8379.000000
4.000000	4.000000	4.000000	60.000000	19999.000000

Num_cols



Cat_cols



Nettoyage des colonnes

Drop car pas d'apport de valeur
(même valeur pour tous les
individus)

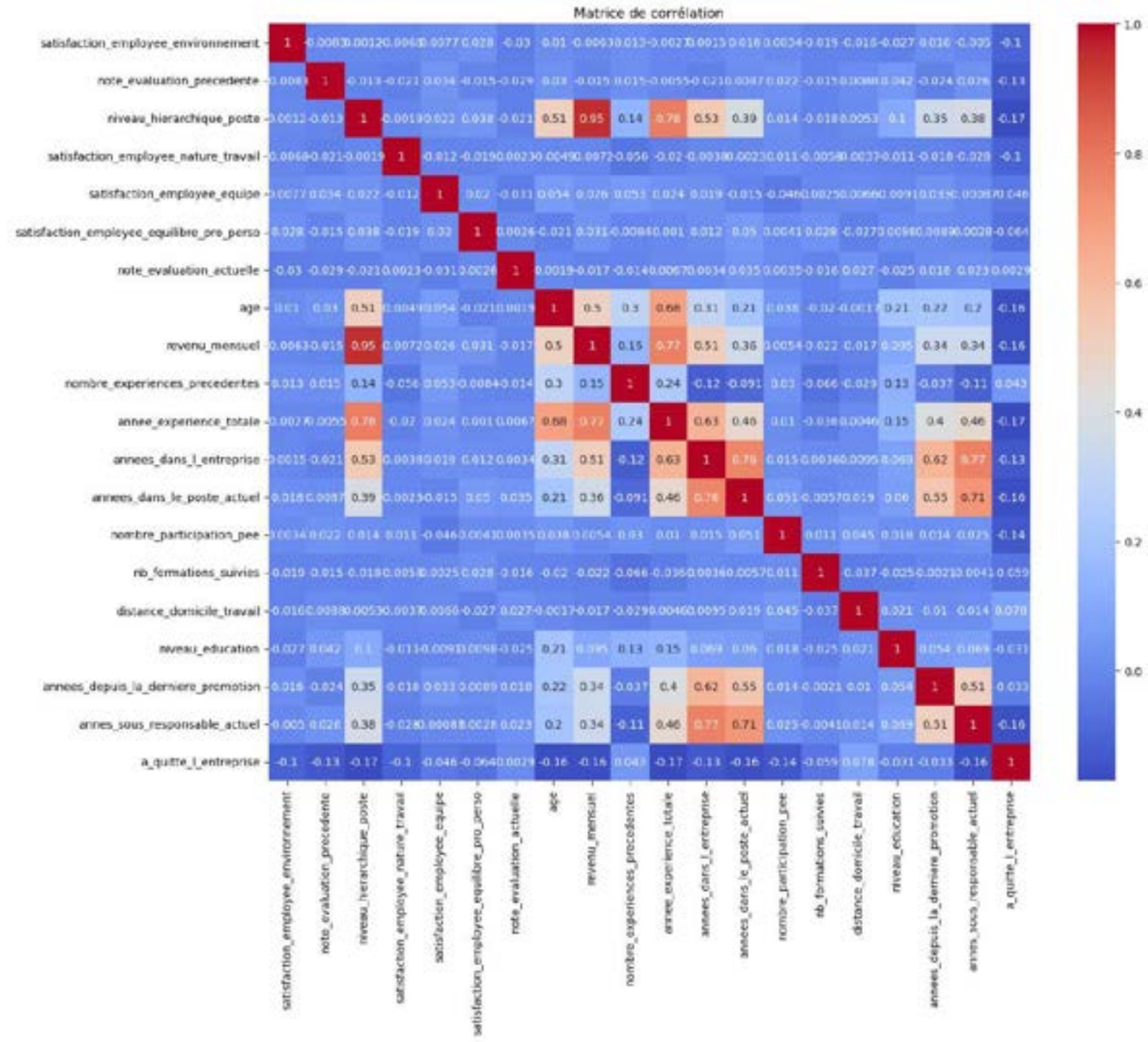
```
1 df['nombre_heures_travailless'].unique()  
] ✓ 0.0s  
  
array([80])
```

```
1 df['nombre_employee_sous_responsabilite'].unique()  
] ✓ 0.0s  
  
array([1])
```

```
1 df.drop(columns=['nombre_employee_sous_responsabilite', 'nombre_heures_travailless'], axis=1, inplace=True)  
] ✓ 0.0s
```

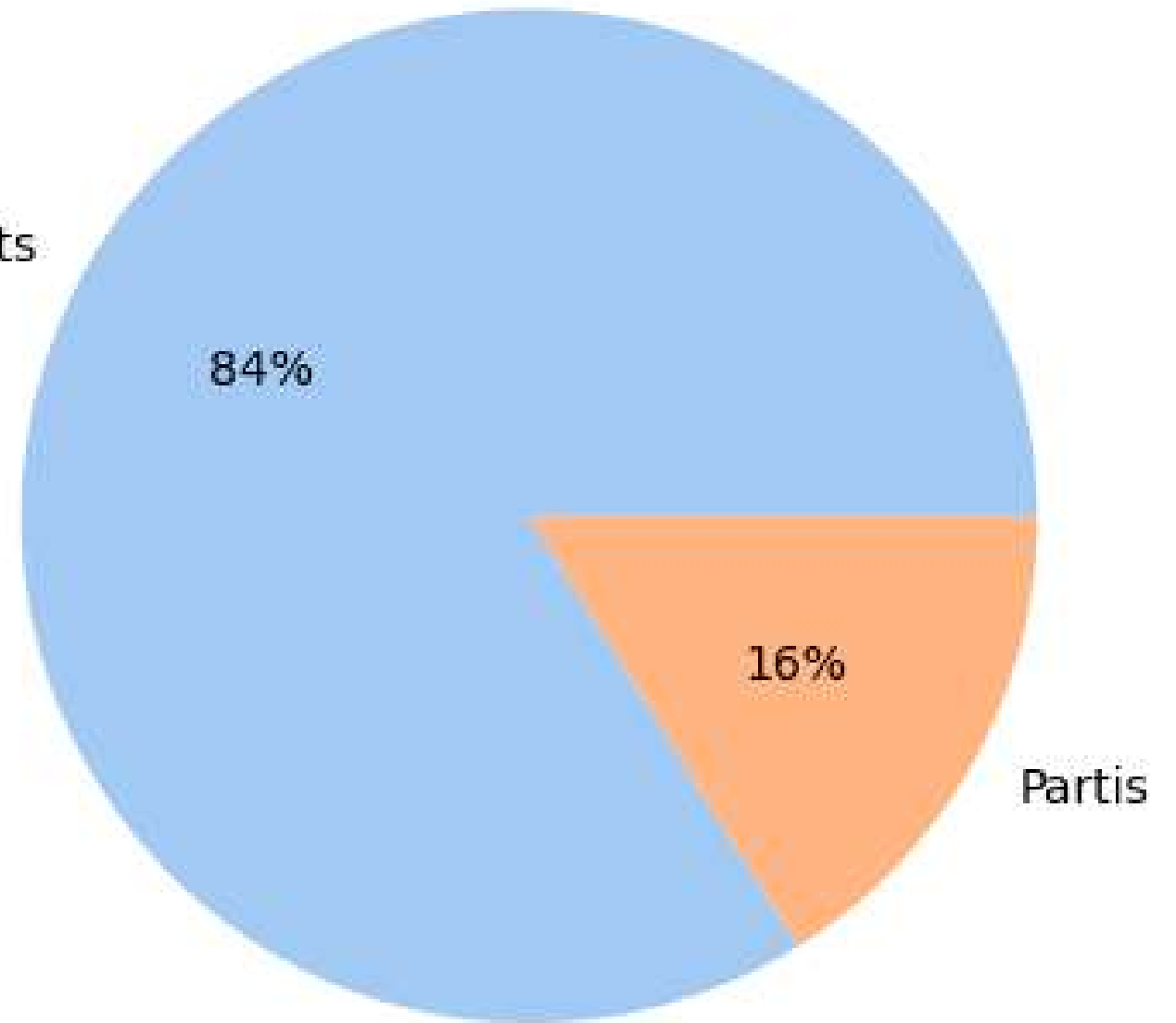
1470 rows × 32 columns --> 1470 rows × 30 columns

Matrice de corrélation



Analyse Target

Restants



Feature Engineering

Feature Engineering & Train-Test split

Train-Test Split

```
1 # Train-Test Split pour éviter le data leakage
2
3 import numpy as np
4 from sklearn.model_selection import train_test_split
5
6 X = df.drop(columns='a_quitte_l_entreprise')
7 y = df['a_quitte_l_entreprise']
8
9
10 X_train, X_test, y_train, y_test = train_test_split(
11     X,
12     y,
13     test_size=0.2,
14     random_state=42,
15     stratify=y
16 )
17
```

c'est un problème de classification où on a un déséquilibre de classes (beaucoup plus de 'non' que de 'oui')
=> on veut la même proportion dans le train et dans le test : "stratify=y" signifie que la séparation respecte la distribution de y.
—> on fera du Stratified KFold Cross-Validation

✓ 0.0s

```
1 print(y.value_counts(normalize=True),
2       y_train.value_counts(normalize=True),
3       y_test.value_counts(normalize=True))
```

✓ 0.0s

```
a_quitte_l_entreprise
0    0.838776
1    0.161224
Name: proportion, dtype: float64 a_quitte_l_entreprise
0    0.838435
1    0.161565
Name: proportion, dtype: float64 a_quitte_l_entreprise
0    0.840136
1    0.159864
Name: proportion, dtype: float64
```

Df : 1470 lignes × 30 colonnes

X_train : 1176 lignes x 29 colonnes

X_test : 294 lignes x 29 colonnes

Création de Features

```
# =====  
# C. Rémunération & Reconnaissance  
# =====  
  
def create_revenu_relatif(df): #  
    df = df.copy()  
  
    median_salary_by_level = (X_train  
                               .groupby("niveau_hierarchique_poste")["revenu_mensuel"]  
                               .median())  
  
    df["revenu_relatif"] = {  
        df["revenu_mensuel"] /  
        df["niveau_hierarchique_poste"].map(median_salary_by_level)  
    }  
  
    return df  
  
def create_sous_paye(df):  
    df["sous_paye"] = (df["revenu_relatif"] < 0.9).astype(int)  
    return df  
  
def create_augmentation_pct(df):  
    df = df.copy()  
  
    df["augmentation_pct"] = {  
        df["augmentation_salaire_precedente"]  
        .str.replace(" %", "", regex=False)  
        .astype(int)  
    }  
  
    return df  
  
def create_faible_augmentation(df):  
    df = df.copy()  
  
    df["faible_augmentation"] = (df["augmentation_pct"] <= 12).astype(int)  
  
    return df
```

```
# =====  
# A. Satisfaction & Performance  
# =====  
  
def create_satisfaction_globale(df):  
    df = df.copy() # Pour éviter de modifier directement l'original  
    df["satisfaction_globale"] = df[  
        {  
            "satisfaction_employe_environnement",  
            "satisfaction_employe_nature_travail",  
            "satisfaction_employe_equipe",  
            "satisfaction_employe_equilibre_pro_perso"  
        }  
    ].mean(axis=1) # moyenne colonne par colonne, ligne par ligne  
    return df  
  
def create_gap_perf_satisfaction(df):  
    df["gap_perf_satisfaction"] = {  
        df["note_evaluation_actuelle"] - df["satisfaction_globale"]  
    }  
    return df  
  
def create_evolution_performance(df):  
    df = df.copy()  
  
    df["evolution_performance"] = {  
        df["note_evaluation_actuelle"] - df["note_evaluation_precedente"]  
    }  
  
    return df  
  
def create_haut_performer(df):  
    df = df.copy()  
  
    df["haut_performer"] = (df["note_evaluation_actuelle"] >= 4).astype(int)  
  
    return df  
  
def create_pee_par_an(df):  
    df = df.copy()  
  
    df["pee_par_an"] = {  
        df["nombre_participation_pee"] /  
        (df["annees_dans_l_entreprise"] + 1)  
    }  
  
    return df
```

```
# =====  
# B. Carrière & Progression  
# =====  
  
def create_ratio_posteanciennete(df):  
    df = df.copy()  
  
    df["ratio_posteanciennete"] = {  
        df["annees_dans_le_poste_actuel"] /  
        (df["annees_dans_l_entreprise"] + 1)  
    }  
    return df  
  
def create_blocage_promotion(df):  
    df = df.copy()  
  
    df["blocage_promotion"] = {  
        df["annees_depuis_la_derniere_promotion"] > 3  
    }.astype(int)  
  
    return df  
  
def create_ratio_manager(df):  
    df = df.copy()  
  
    df["ratio_manager"] = {  
        df["annees_sous_responsable_actuel"] /  
        (df["annees_dans_l_entreprise"] + 1)  
    }  
  
    return df  
  
def create_formation_par_an(df):  
    df = df.copy()  
  
    df["formation_par_an"] = {  
        df["nb_formations_suivies"] /  
        (df["annees_dans_l_entreprise"] + 1)  
    }  
  
    return df
```

Création de Features

```
# =====  
# D. Équilibre vie pro / contraintes  
# =====  
  
def create_jeune(df):  
    df = df.copy()  
  
    df["jeune"] = (df["age"] < 30).astype(int)  
  
    return df  
  
def create_heures_suppl(df):  
    df = df.copy()  
  
    df["heures_suppl"] = (df["heure_supplementaires"] == "Oui").astype(int)  
  
    return df  
  
def create_distance_penible(df):  
    df = df.copy()  
  
    threshold = X_train["distance_domicile_travail"].quantile(0.75) # un paramètre qui doit être fit sur X_train seulement  
  
    df["distance_penible"] = (  
        df["distance_domicile_travail"] > threshold  
    ).astype(int)  
  
    return df  
  
def create_mobilite_forte(df):  
    df = df.copy()  
  
    df["mobilite_forte"] = (  
        df["frequence_deplacement"] == "Frequent"  
    ).astype(int)  
  
    return df
```

```
def feature_engineering(df):
```

```
    df = create_satisfaction_globale(df)  
    df = create_gap_perf_satisfaction(df)  
    df = create_evolution_performance(df)  
    df = create_haut_performer(df)  
    df = create_pee_par_an(df)
```

```
    df = create_ratio_poste_anciennete(df)  
    df = create_blocage_promotion(df)  
    df = create_ratio_manager(df)  
    df = create_formation_par_an(df)
```

```
    df = create_revenu_relatif(df)  
    df = create_sous_paye(df)  
    df = create_augmentation_pct(df)  
    df = create_faible_augmentation(df)
```

```
    df = create_jeune(df)  
    df = create_heures_suppl(df)  
    df = create_distance_penible(df)  
    df = create_mobilite_forte(df)
```

```
    return df
```

```
X_train = feature_engineering(X_train)
```

```
X_test = feature_engineering(X_test) # application des mêmes transformations sur le test set
```

Création de Features

```
1 X_train.info()
✓ 0.0s

<class 'pandas.core.frame.DataFrame'>
Index: 1176 entries, 1194 to 1292
Data columns (total 46 columns):
#   Column                                                                 Non-Null Count  Dtype
---  -
0   id_employe                                                            1176 non-null   object
1   satisfaction_employe_environnement                                   1176 non-null   int64
2   note_evaluation_precedente                                           1176 non-null   int64
3   niveau_hierarchique_poste                                            1176 non-null   int64
4   satisfaction_employe_nature_travail                                  1176 non-null   int64
5   satisfaction_employe_equipe                                           1176 non-null   int64
6   satisfaction_employe_equilibre_pro_perso                             1176 non-null   int64
7   note_evaluation_actuelle                                              1176 non-null   int64
8   heure_supplementaires                                                 1176 non-null   object
9   augmentation_salaire_precedente                                       1176 non-null   object
10  age                                                                    1176 non-null   int64
11  genre                                                                  1176 non-null   object
12  revenu_mensuel                                                         1176 non-null   int64
13  statut_marital                                                         1176 non-null   object
14  departement                                                            1176 non-null   object
15  poste                                                                  1176 non-null   object
16  nombre_experiences_precedentes                                         1176 non-null   int64
17  annee_experience_totale                                                 1176 non-null   int64
18  annees_dans_l_entreprise                                               1176 non-null   int64
19  annees_dans_le_poste_actuel                                            1176 non-null   int64
...
44  distance_penible                                                       1176 non-null   int64
45  mobilite_forte                                                         1176 non-null   int64
dtypes: float64(7), int64(29), object(10)
memory usage: 431.8+ KB
```

Train-Test
Split

Df : 1470 lignes × 30 colonnes



X_train : 1176 lignes x 29 colonnes

X_test : 294 lignes x 29 colonnes

Création
de features

1176 lignes x 46 colonnes

294 lignes x 46 colonnes

Nettoyage de colonnes

```
# Colonnes à supprimer après Feature Engineering & justifications

cols_supp = ['id_employee',                # les ID sont uniques et n'apportent pas d'information
             'ayant_enfants',             # même valeur pour tous donc non-informatif
             # Doublons d'information
             'revenu_mensuel',             # information biaisée si non regroupée par niveau hiérarchique de poste + est déjà contenue dans 'revenu_relatif'
             'augmentation_salaire_precedente', # info déjà dans 'augmentation_pct', plus exploitable par le modèle
             'heure_supplementaires'       # info déjà dans 'heures_supp', plus exploitable par le modèle
            ]

X_train = X_train.drop(columns=cols_supp)
with pd.option_context('display.max_columns', None):
    display(X_train.head(10))

# Suppression des mêmes colonnes sur le X_test
X_test = X_test.drop(columns=cols_supp)
```

X_train : 1176 lignes x 46 colonnes → 1176 lignes x 41 colonnes

X_test : 294 lignes x 46 colonnes → 294 lignes x 41 colonnes

Valeurs Manquantes

```
1 summary = pd.DataFrame({
2     "dtype": X_train.dtypes,
3     "n_unique": X_train.nunique(),
4     "missing": X_train.isna().sum()
5 })
6
7 summary.sort_values("n_unique")
```

✓ 0.0s

	dtype	n_unique	missing
ayant_enfants	object	1	0
mobilite_forte	int64	2	0
sous_paye	int64	2	0
blocage_promotion	int64	2	0
distance_penible	int64	2	0
jeune	int64	2	0
genre	object	2	0
heures_supp	int64	2	0
faible_augmentation	int64	2	0
haut_performer	int64	2	0
note_evaluation_actuelle	int64	2	0
heure_supplementaires	object	2	0
statut_marital	object	3	0
departement	object	3	0
frequence_deplacement	object	3	0
satisfaction_employe_equilibre_pro_perso	int64	4	0
satisfaction_employe_equipe	int64	4	0
satisfaction_employe_environnement	int64	4	0

Attribution des types de données appropriés

Application officielle des transformations de type de données à X_train et X_test

```
1 import pandas as pd
2
3 def Feature_Engineering(df):
4     df = df.copy()
5
6     n_rows = len(df)
7
8     for col in df.columns:
9         current_dtype = df[col].dtype
10        n_unique = df[col].nunique(dropna=False)
11
12        if n_unique == n_rows:
13            df.drop(columns=col, inplace=True)
14
15        elif (
16            n_unique in [1, 2]
17            and set(df[col].dropna().unique()).issubset({0, 1})
18            and col != "note_evaluation_actuelle"
19        ):
20            df[col] = df[col].astype("bool")
21
22        elif current_dtype in ["int64", "float64"] and n_unique < 15:
23            df[col] = df[col].astype("int64")
24
25        elif current_dtype in ["int64", "float64"]:
26            df[col] = df[col].astype("float64")
27
28        else:
29            df[col] = df[col].astype("category")
30
31    return df
32
33 X_train = Feature_Engineering(X_train)
34 X_test = Feature_Engineering(X_test)
```

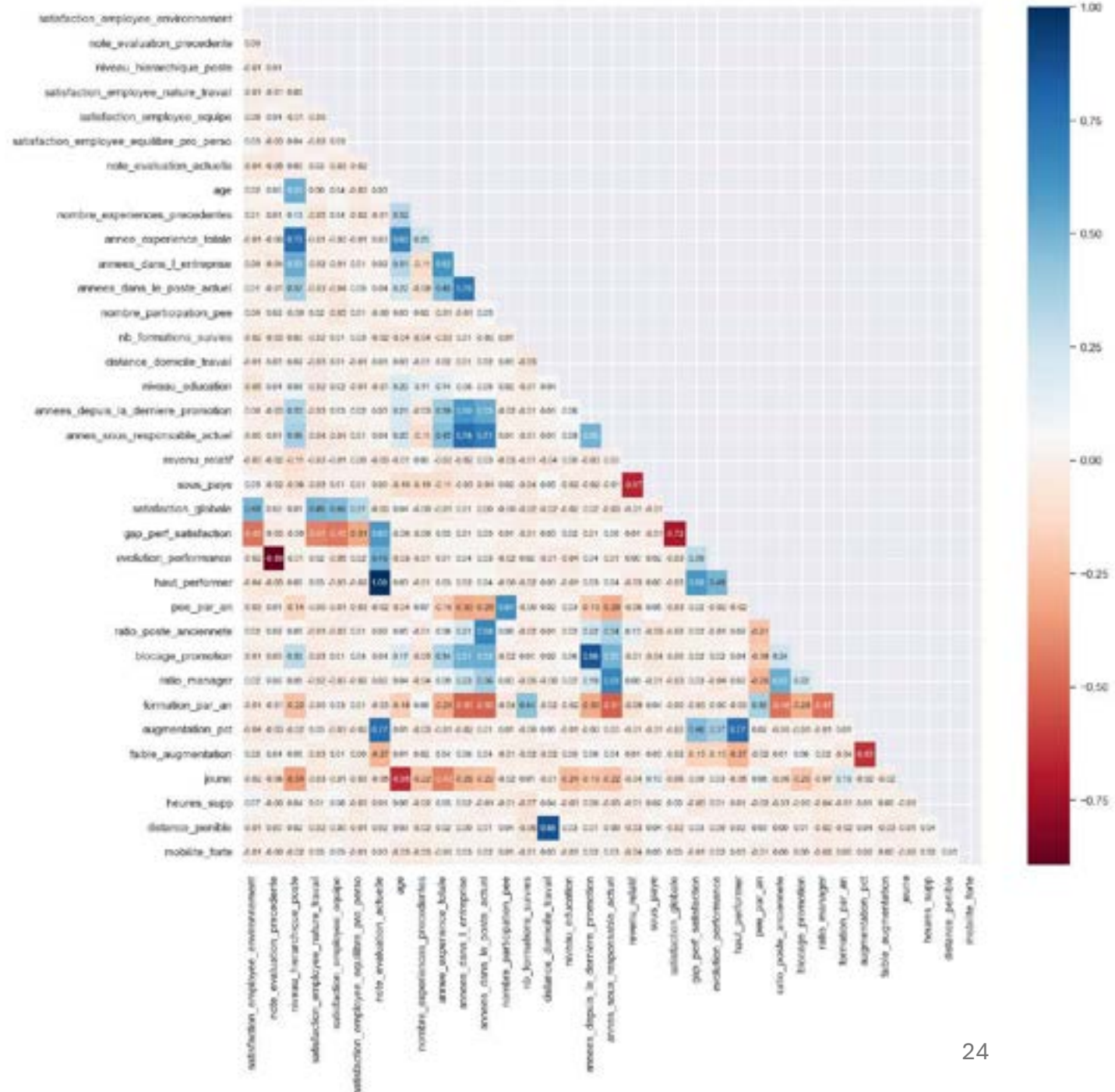
Matrice de corrélation

Sur les num_cols

```
1 import numpy as np
2
3 threshold = 0.8 # seuil de corrélation élevée
4
5 # Triangle supérieur (ignore doublons et diagonale)
6 upper_triangle = corr_matrix.where(
7     np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
8 )
9
10 # Paires corrélées au-dessus du seuil
11 high_corr_pairs = [
12     (col, row, upper_triangle.loc[row, col])
13     for col in upper_triangle.columns
14     for row in upper_triangle.index
15     if abs(upper_triangle.loc[row, col]) > threshold
16 ]
17
18 high_corr_pairs
19
20 print("Paires de variables fortement corrélées (> 0.8 ou < -0.8) :\n")
21 for var1, var2, corr_val in high_corr_pairs:
22     print(f"{var1} et {var2} : corrélation = {corr_val:.2f}")
23
```

Paires de variables fortement corrélées (> 0.8 ou < -0.8) :

evolution_performance et note_evaluation_precedente : corrélation = -0.89
haut_performer et note_evaluation_actuelle : corrélation = 1.00
blocage_promotion et annees_depuis_la_derniere_promotion : corrélation = 0.86
distance_pénible et distance_domicile_travail : corrélation = 0.88



Préparation à la modélisation

1. Encodage & Préparation de pipelines

```
# fonction pour préparer les colonnes en fonction du jeu choisi
def get_col_types(df):
    cat_cols = df.select_dtypes(include=['category']).columns.tolist()
    num_cols = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
    bool_cols = df.select_dtypes(include=['bool']).columns.tolist()
    return cat_cols, num_cols, bool_cols
```

```
# fonction pour construire un pipeline à partir d'un DataFrame et d'un modèle
def build_pipeline(X, model, sampler=None):
    cat_cols, num_cols, bool_cols = get_col_types(X)

    # Pipelines pour chaque type
    numeric_transformer = Pipeline([
        ('imputer', SimpleImputer(strategy='median')), # on intègre cette étape essentielle normalement, même si dans notre cas, il n'y a aucune valeur manquante
        ('scaler', StandardScaler())
    ])

    categorical_transformer = Pipeline([
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore', drop='first')) # drop='first' enlève une colonne pour éviter la colinéarité
    ])

    # On combine les transformations
    transformers=[
        ('num', numeric_transformer, num_cols),
        ('cat', categorical_transformer, cat_cols),
    ]

    # On ajoute au transformer la transformation "passthrough" uniquement si 'bool_cols' n'est pas vide
    if len(bool_cols) > 0:
        transformers.append(('bool', 'passthrough', bool_cols))

    preprocessor = ColumnTransformer(
        transformers=transformers) # transformers contient la transformation des 'bool_cols' ou pas.

    # Pipeline selon la méthode de gestion du déséquilibre des classes
    # MÉTHODE 0 : Définition d'un cas Dummy (modèle le plus simpliste) qui sert de baseline
    if sampler == 'dummy':
        pipe = Pipeline([
            ('preprocessor', preprocessor),
            ('model', model)
        ])

    # MÉTHODE 1 : Gestion du class weight intégrée dans le modèle
    elif sampler == 'class_weight':
        pipe = Pipeline([
            ('preprocessor', preprocessor),
            ('model', model)
        ])

    # MÉTHODE 2 : Oversampling SMOTE (Synthetic Minority Oversampling Technique)
    # une technique statistique permettant d'augmenter le nombre de cas d'un jeu de données de façon équilibrée
    elif sampler == 'smote':
        pipe = ImbPipeline([
            ('preprocessor', preprocessor),
            ('smote', SMOTE(random_state=42)),
            ('model', model)
        ])

    # MÉTHODE 3 : Undersampling (sous-échantillonnage)
    else:
        pipe = ImbPipeline([
            ('preprocessor', preprocessor),
            ('undersample', RandomUnderSampler(random_state=42)),
            ('model', model)
        ])

    return pipe
```


1. Préparation des modèles avec traitement du déséquilibre des classes

Choix métriques

Cross validation

```
# Liste de modèles à utiliser ---
# 1. Modèles pour la MÉTHODE 1 du class_weight intégré
models_class_weight = {
    "logistic": LogisticRegression(
        class_weight="balanced",
        max_iter=1000,
        random_state=42
    ),
    "random_forest": RandomForestClassifier(
        class_weight="balanced",
        n_estimators=200,
        random_state=42,
        n_jobs=-1
    ),
    "lightgbm": LGBMClassifier(
        class_weight="balanced",
        n_estimators=200,
        learning_rate=0.05,
        random_state=42,
        verbose=-1,
        n_jobs=-1
    )
}

# 2. Modèles pour les MÉTHODES 2 et 3 de suréchantillonnage et de sous-échantillonnage
models = {
    "logistic": LogisticRegression(
        max_iter=1000,
        random_state=42
    ),
    "random_forest": RandomForestClassifier(
        n_estimators=200,
        random_state=42,
        n_jobs=-1
    ),
    "lightgbm": LGBMClassifier(
        n_estimators=200,
        learning_rate=0.05,
        random_state=42,
        n_jobs=-1,
        verbose=-1
    )
}

# Liste des samplers
sampling_methods = ["class_weight", "smote", "undersample"]

# --- Préparation de la Cross-Validation ---
scoring = ["precision", "recall", "f1", "roc_auc"]

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# --- Préparation BOUCLE FINALE ---
# appliquent chaque méthode de sampling et sur chacun des 3 modèles
# donnant des scores F1, Recall, et ROC-AUC

results = [] # Initialisation d'une liste dans laquelle stocker le résultat des scores f1, recall, ROC-AUC
```

Dummy Classifier

```
for method in sampling_methods:

    if method == "class_weight":
        model_dict = models_class_weight
    else:
        model_dict = models

    for model_name, model in model_dict.items():

        if model_name == "logistic":
            X_input = X_train[features_set_regression] # sans les colonnes colinéaires
        else:
            X_input = X_train[features_set_general] # toutes les features

        pipeline = build_pipeline(X_input, model, method)

        scores = cross_validate(
            pipeline,
            X_input,
            y_train,
            cv=cv,
            scoring=scoring,
            verbose=0 # pour éviter les logs sklearn trop longs
        )

        # Calcul des moyennes et écarts-types des scores
        mean_scores = {f"({metric})_mean": scores[f'test_{metric}'].mean() for metric in scoring}
        std_scores = {f"({metric})_std": scores[f'test_{metric}'].std() for metric in scoring}

        # Ajout d'un dictionnaire avec toutes les infos dans la liste results
        results.append({
            'method': method,
            'model': model_name,
            **mean_scores, # dépile le dictionnaire des scores
            **std_scores
        })

# CAS DUMMY
dummy = DummyClassifier(strategy="most_frequent")

scoring_with_accuracy = ["precision", "recall", "f1", "roc_auc", "accuracy"]

pipeline = build_pipeline(X_train[features_set_general], dummy, sampler="dummy")

scores = cross_validate(
    pipeline,
    X_train[features_set_general],
    y_train,
    cv=cv,
    scoring=scoring_with_accuracy
)

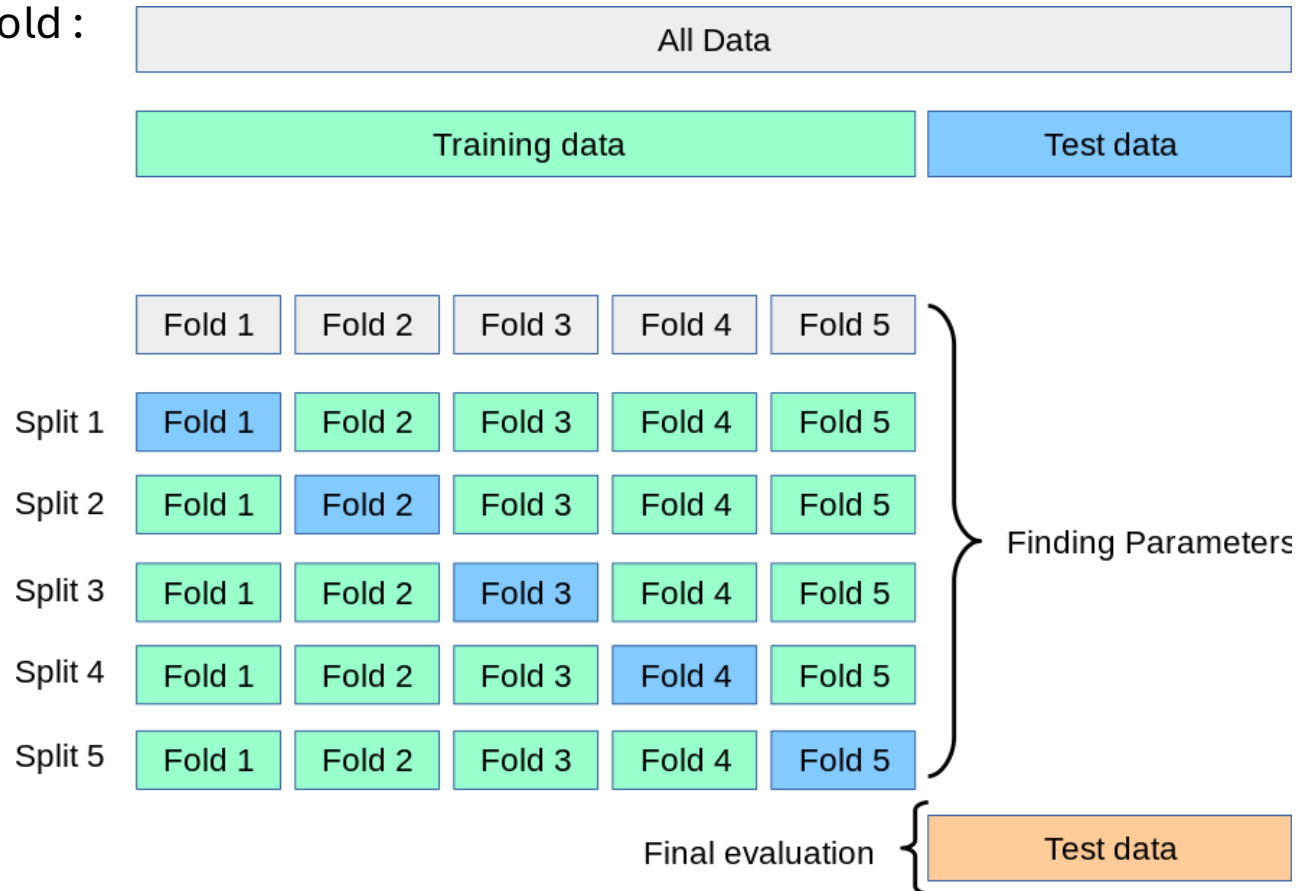
mean_scores = {f"({metric})_mean": scores[f'test_{metric}'].mean() for metric in scoring}
std_scores = {f"({metric})_std": scores[f'test_{metric}'].std() for metric in scoring}

results.append({
    'method': 'none',
    'model': 'dummy',
    **mean_scores,
    **std_scores
})
```


A. Cross-validation

Cross validation

Kfold :



Plusieurs types de Cross-validation :

- Kfold
- Stratified <--- Celui qu'on utilise
- Temporel
- etc.

Choix du meilleur modèle : Evaluation des scores de la Cross-Validation

B. Choix de métriques d'évaluation

Pour rappel :

- **Précision (Precision)** : proportion de prédictions positives correctes parmi toutes les prédictions positives

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Rappel (Recall)** : proportion de vrais positifs correctement identifiés parmi tous les positifs réels

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-score** : moyenne harmonique entre la précision et le rappel

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **Accuracy** : proportion de prédictions correctes parmi toutes les observations

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

C. Dummy Classifier

```
1 # Scores du Dummy Classifier
2 scores
```

✓ 0.0s

```
{'fit_time': array([0.00751829, 0.00736785, 0.00854707, 0.00960422, 0.0071578 ]),
'score_time': array([0.00918293, 0.00885606, 0.0117991 , 0.00955176, 0.00932789]),
'test_precision': array([0., 0., 0., 0., 0.]),
'test_recall': array([0., 0., 0., 0., 0.]),
'test_f1': array([0., 0., 0., 0., 0.]),
'test_roc_auc': array([0.5, 0.5, 0.5, 0.5, 0.5]),
'test_accuracy': array([0.83898305, 0.83829787, 0.83829787, 0.83829787, 0.83829787])}
```

Le Dummy Classifier prédit toujours la classe majoritaire du jeu de données.

Dans ce contexte, le score Accuracy est élevé (~0.84), mais il est trompeur et non pertinent pour cette problématique de churn déséquilibrée. En effet, il peut obtenir une bonne accuracy sans réussir à identifier les clients qui partent.

En revanche, les autres métriques sont très faibles :

- Recall = 0 : aucun client churn n'est détecté
- Precision = 0 : aucune prédiction positive correcte
- F1-score = 0 : performance nulle sur la classe minoritaire
- ROC-AUC = 0.5 : équivalent à une prédiction aléatoire

Ce modèle sert uniquement de baseline pour comparer les performances des autres modèles.

1. Préparation des modèles avec traitement du déséquilibre des classes

Choix métriques

Cross validation

```
# Liste de modèles à utiliser ---
# 1. Modèles pour la MÉTHODE 1 du class_weight intégré
models_class_weight = {
    "logistic": LogisticRegression(
        class_weight="balanced",
        max_iter=1000,
        random_state=42
    ),
    "random_forest": RandomForestClassifier(
        class_weight="balanced",
        n_estimators=200,
        random_state=42,
        n_jobs=-1
    ),
    "lightgbm": LGBMClassifier(
        class_weight="balanced",
        n_estimators=200,
        learning_rate=0.05,
        random_state=42,
        verbose=-1,
        n_jobs=-1
    )
}

# 2. Modèles pour les MÉTHODES 2 et 3 de suréchantillonnage et de sous-échantillonnage
models = {
    "logistic": LogisticRegression(
        max_iter=1000,
        random_state=42
    ),
    "random_forest": RandomForestClassifier(
        n_estimators=200,
        random_state=42,
        n_jobs=-1
    ),
    "lightgbm": LGBMClassifier(
        n_estimators=200,
        learning_rate=0.05,
        random_state=42,
        n_jobs=-1,
        verbose=-1
    )
}

# Liste des samplers
sampling_methods = ["class_weight", "smote", "undersample"]

# --- Préparation de la Cross-Validation ---
scoring = ["precision", "recall", "f1", "roc_auc"]

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# --- Préparation BOUCLE FINALE ---
# appliquent chaque méthode de sampling et sur chacun des 3 modèles
# donnant des scores F1, Recall, et ROC-AUC

results = [] # initialisation d'une liste dans laquelle stocker le résultat des scores f1, recall, ROC-AUC
```

Dummy Classifier

```
for method in sampling_methods:

    if method == "class_weight":
        model_dict = models_class_weight
    else:
        model_dict = models

    for model_name, model in model_dict.items():

        if model_name == "logistic":
            X_input = X_train[features_set_regression] # sans les colonnes colinéaires
        else:
            X_input = X_train[features_set_general] # toutes les features

        pipeline = build_pipeline(X_input, model, method)

        scores = cross_validate(
            pipeline,
            X_input,
            y_train,
            cv=cv,
            scoring=scoring,
            verbose=0 # pour éviter les logs sklearn trop longs
        )

        # Calcul des moyennes et écarts-types des scores
        mean_scores = {f'{metric}_mean': scores[f'test_{metric}'].mean() for metric in scoring}
        std_scores = {f'{metric}_std': scores[f'test_{metric}'].std() for metric in scoring}

        # Ajout d'un dictionnaire avec toutes les infos dans la liste results
        results.append({
            'method': method,
            'model': model_name,
            **mean_scores, # dépile le dictionnaire des scores
            **std_scores
        })

# CAS DUMMY
dummy = DummyClassifier(strategy="most_frequent")

scoring_with_accuracy = ["precision", "recall", "f1", "roc_auc", "accuracy"]

pipeline = build_pipeline(X_train[features_set_general], dummy, sampler="dummy")

scores = cross_validate(
    pipeline,
    X_train[features_set_general],
    y_train,
    cv=cv,
    scoring = scoring_with_accuracy
)

mean_scores = {f'{metric}_mean': scores[f'test_{metric}'].mean() for metric in scoring}
std_scores = {f'{metric}_std': scores[f'test_{metric}'].std() for metric in scoring}

results.append({
    'method': 'none',
    'model': 'dummy',
    **mean_scores,
    **std_scores
})
```

2. Choix du meilleur modèle : Evaluation des scores de la Cross-Validation

```
1 # Tri des résultats par score ROC-AUC_mean décroissant (métrique la plus intéressante avec le F1-score)
2 scores_df.sort_values(by='roc_auc_mean', ascending=False)
3 ✓ 0.0s
```

	method	model	precision_mean	recall_mean	f1_mean	roc_auc_mean	precision_std	recall_std	f1_std	roc_auc_std
3	smote	logistic	0.429959	0.705263	0.530632	0.836939	0.050237	0.053674	0.030917	0.020498
0	class_weight	logistic	0.409425	0.742105	0.523598	0.835905	0.046894	0.060925	0.025254	0.021980
6	undersample	logistic	0.380289	0.747368	0.502449	0.824547	0.035125	0.039386	0.027800	0.020251
4	smote	random_forest	0.792563	0.289474	0.414153	0.817985	0.126099	0.095610	0.108713	0.041849
2	class_weight	lightgbm	0.621698	0.447368	0.513705	0.807973	0.084472	0.086483	0.070091	0.037189
5	smote	lightgbm	0.681854	0.352632	0.461489	0.803992	0.090541	0.063595	0.066566	0.043094
1	class_weight	random_forest	0.833333	0.163158	0.264792	0.796368	0.155036	0.065314	0.091112	0.037565
8	undersample	lightgbm	0.324338	0.684211	0.439239	0.788154	0.017354	0.070613	0.026125	0.025521
7	undersample	random_forest	0.346781	0.657895	0.454060	0.774193	0.023178	0.060009	0.033721	0.030297
9	none	dummy	0.000000	0.000000	0.000000	0.500000	0.000000	0.000000	0.000000	0.000000

```
1 # Tri par rapport au score F1_mean
2 scores_df.sort_values(by='f1_mean', ascending=False)
3 ✓ 0.0s
```

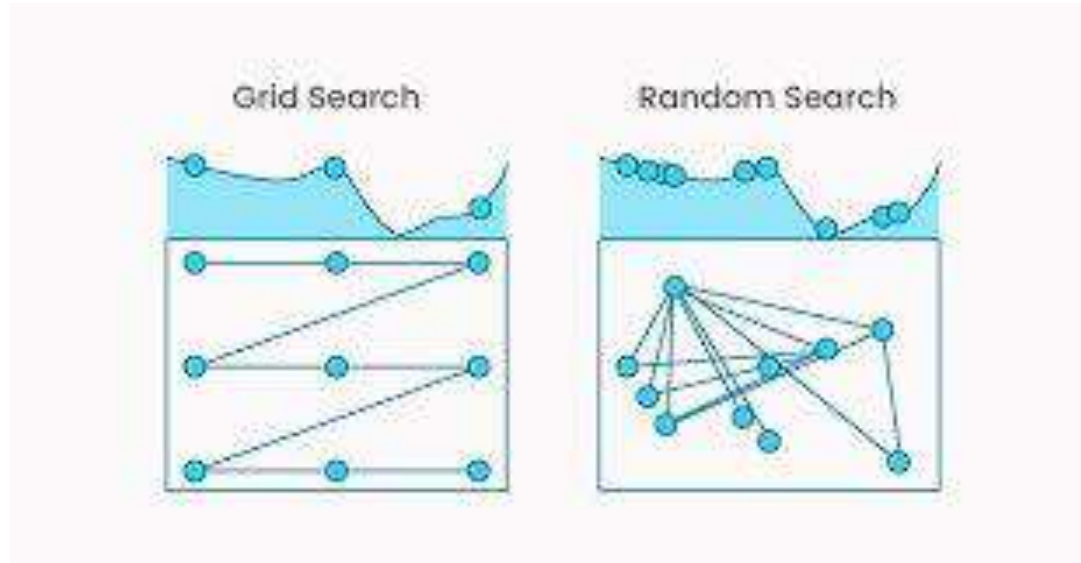
	method	model	precision_mean	recall_mean	f1_mean	roc_auc_mean	precision_std	recall_std	f1_std	roc_auc_std
3	smote	logistic	0.429959	0.705263	0.530632	0.836939	0.050237	0.053674	0.030917	0.020498
0	class_weight	logistic	0.409425	0.742105	0.523598	0.835905	0.046894	0.060925	0.025254	0.021980
2	class_weight	lightgbm	0.621698	0.447368	0.513705	0.807973	0.084472	0.086483	0.070091	0.037189
6	undersample	logistic	0.380289	0.747368	0.502449	0.824547	0.035125	0.039386	0.027800	0.020251
5	smote	lightgbm	0.681854	0.352632	0.461489	0.803992	0.090541	0.063595	0.066566	0.043094
7	undersample	random_forest	0.346781	0.657895	0.454060	0.774193	0.023178	0.060009	0.033721	0.030297
8	undersample	lightgbm	0.324338	0.684211	0.439239	0.788154	0.017354	0.070613	0.026125	0.025521
4	smote	random_forest	0.792563	0.289474	0.414153	0.817985	0.126099	0.095610	0.108713	0.041849
1	class_weight	random_forest	0.833333	0.163158	0.264792	0.796368	0.155036	0.065314	0.091112	0.037565
9	none	dummy	0.000000	0.000000	0.000000	0.500000	0.000000	0.000000	0.000000	0.000000

```
1 # Tri par rapport au score Recall_mean (métrique importante après ROC-AUC et F1)
2 scores_df.sort_values(by='recall_mean', ascending=False)
3 ✓ 0.0s
```

	method	model	precision_mean	recall_mean	f1_mean	roc_auc_mean	precision_std	recall_std	f1_std	roc_auc_std
6	undersample	logistic	0.380289	0.747368	0.502449	0.824547	0.035125	0.039386	0.027800	0.020251
0	class_weight	logistic	0.409425	0.742105	0.523598	0.835905	0.046894	0.060925	0.025254	0.021980
3	smote	logistic	0.429959	0.705263	0.530632	0.836939	0.050237	0.053674	0.030917	0.020498
8	undersample	lightgbm	0.324338	0.684211	0.439239	0.788154	0.017354	0.070613	0.026125	0.025521
7	undersample	random_forest	0.346781	0.657895	0.454060	0.774193	0.023178	0.060009	0.033721	0.030297
2	class_weight	lightgbm	0.621698	0.447368	0.513705	0.807973	0.084472	0.086483	0.070091	0.037189
5	smote	lightgbm	0.681854	0.352632	0.461489	0.803992	0.090541	0.063595	0.066566	0.043094
4	smote	random_forest	0.792563	0.289474	0.414153	0.817985	0.126099	0.095610	0.108713	0.041849
1	class_weight	random_forest	0.833333	0.163158	0.264792	0.796368	0.155036	0.065314	0.091112	0.037565
9	none	dummy	0.000000	0.000000	0.000000	0.500000	0.000000	0.000000	0.000000	0.000000

3. Recherche d'hyperparamètres

- Recherche d'hyperparamètres avec GridSearchCV
- RandomSearchCV



Exemples : LightGBM

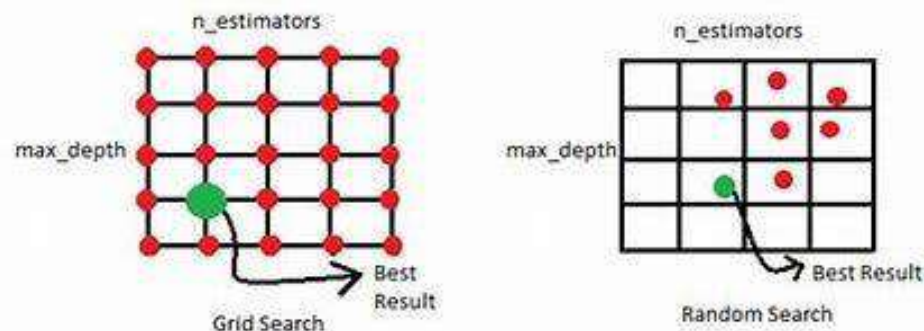
```
# Petite grille
lgb_param_grid = {
    "model__n_estimators": [100, 300],
    "model__learning_rate": [0.05, 0.1],
    "model__max_depth": [-1, 10]
}
```

Random Forest

```
# 3. GridSearch (sur le train uniquement)
param_grid_rf = {
    "model__n_estimators": [200, 500],
    "model__max_depth": [None, 10],
    "model__min_samples_leaf": [1, 5]
}

grid_rf = GridSearchCV(
    pipe_rf,
    param_grid=param_grid_rf,
    cv=3,
    scoring="r2",
    n_jobs=-1
)

grid_rf.fit(X_train, y_train)
```



GridSearch CV

```
# Modèle choisi
logreg = LogisticRegression(
    class_weight='balanced',
    max_iter=1000,
    random_state=42
)

# On construit le pipeline avec le modèle
pipeline = build_pipeline(X_train, logreg, sampler='class_weight')

# Grille d'hyperparamètres à tester
param_grid = {
    'model__C': [0.01, 0.1, 1, 10, 100], # petit C → modèle très régularisé → plus simple → moins de surapprentissage
    'model__penalty': ['l1', 'l2', None], # grand C → modèle moins régularisé → plus flexible → peut overfit
    'model__solver': ['saga'],
}

# Préparation de la recherche par grille avec validation croisée
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    scoring='roc_auc',
    cv=5,
    n_jobs=-1,
    verbose=1
)
```

Fitting 5 folds for each of 15 candidates, totalling 75 fits

Meilleurs paramètres : {'model__C': 0.1, 'model__penalty': 'l2', 'model__solver': 'saga'}

Meilleur score ROC AUC : 0.83015177152233

	method	model	precision_mean	recall_mean	f1_mean	roc_auc_mean	precision_std	recall_std	f1_std	roc_auc_std
0	class_weight	Logistic_Regression	0.404169	0.736842	0.517882	0.83395	0.04666	0.060009	0.0249	0.02356

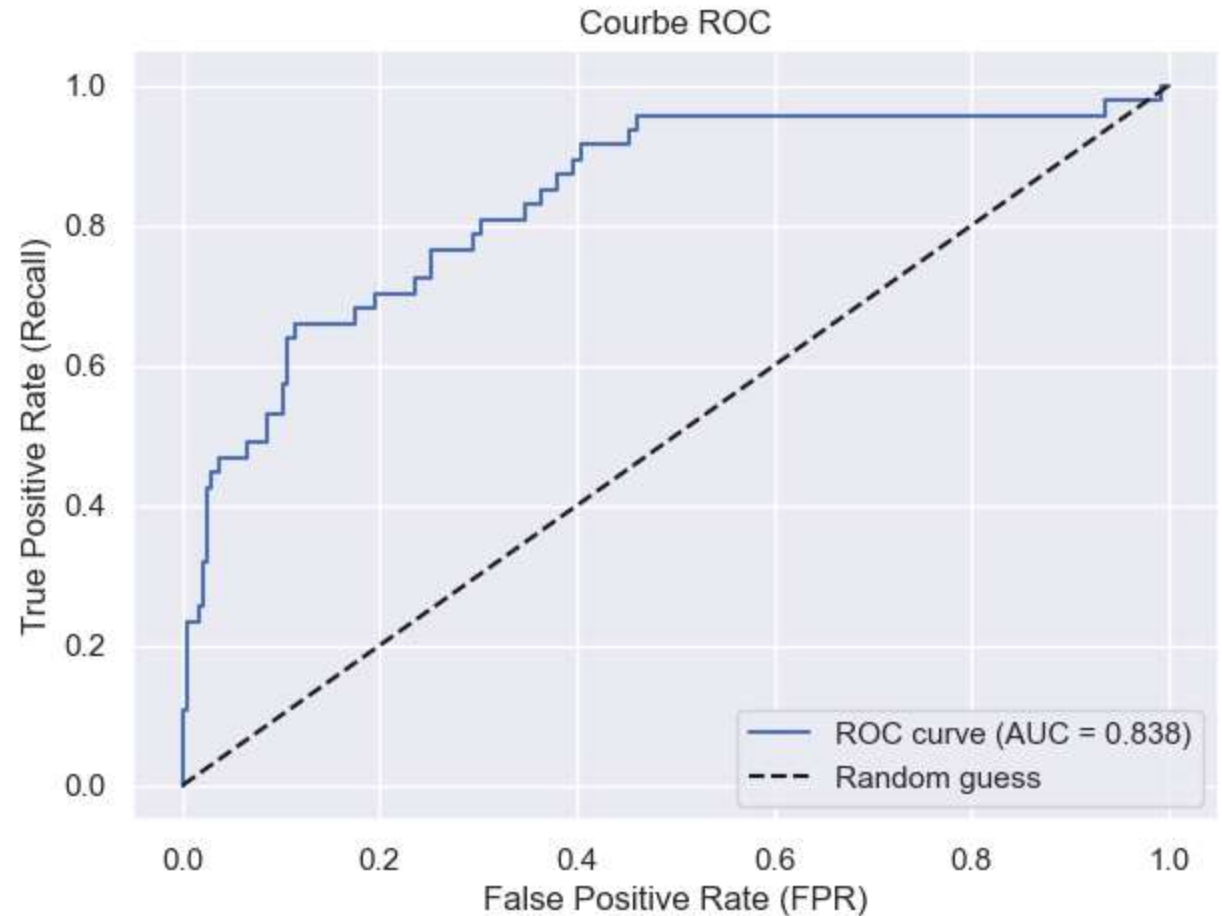
Évaluation finale

Evaluation finale avec les meilleures hyperparamètres & (modèle, méthode)

	precision	recall	f1-score	support
0	0.93	0.83	0.87	247
1	0.42	0.66	0.51	47
accuracy			0.80	294
macro avg	0.67	0.74	0.69	294
weighted avg	0.85	0.80	0.82	294

ROC AUC: 0.8384012404169179

1. Calcul et affichage de la ROC



Seuil optimal

1. Calcul et affichage de la ROC

Nous souhaitons maximiser la **spécificité** (c'est-à-dire la proportion de personnes correctement prédites comme ne partant pas parmi celles qui restent réellement dans l'entreprise) :

$$\text{specificity} = \frac{TN}{TN + FP}$$

Or :

$$FPR = 1 - \text{specificity}$$

Il est donc nécessaire que le **FPR** soit le plus faible possible.

Nous cherchons ainsi un seuil qui maximise le **rappel (TPR)** tout en minimisant le **FPR**.

La maximisation du score de **Youden** permet de trouver le meilleur compromis (détection des vrais positifs et réduction des fausses alertes) :

$$J = TPR - FPR$$

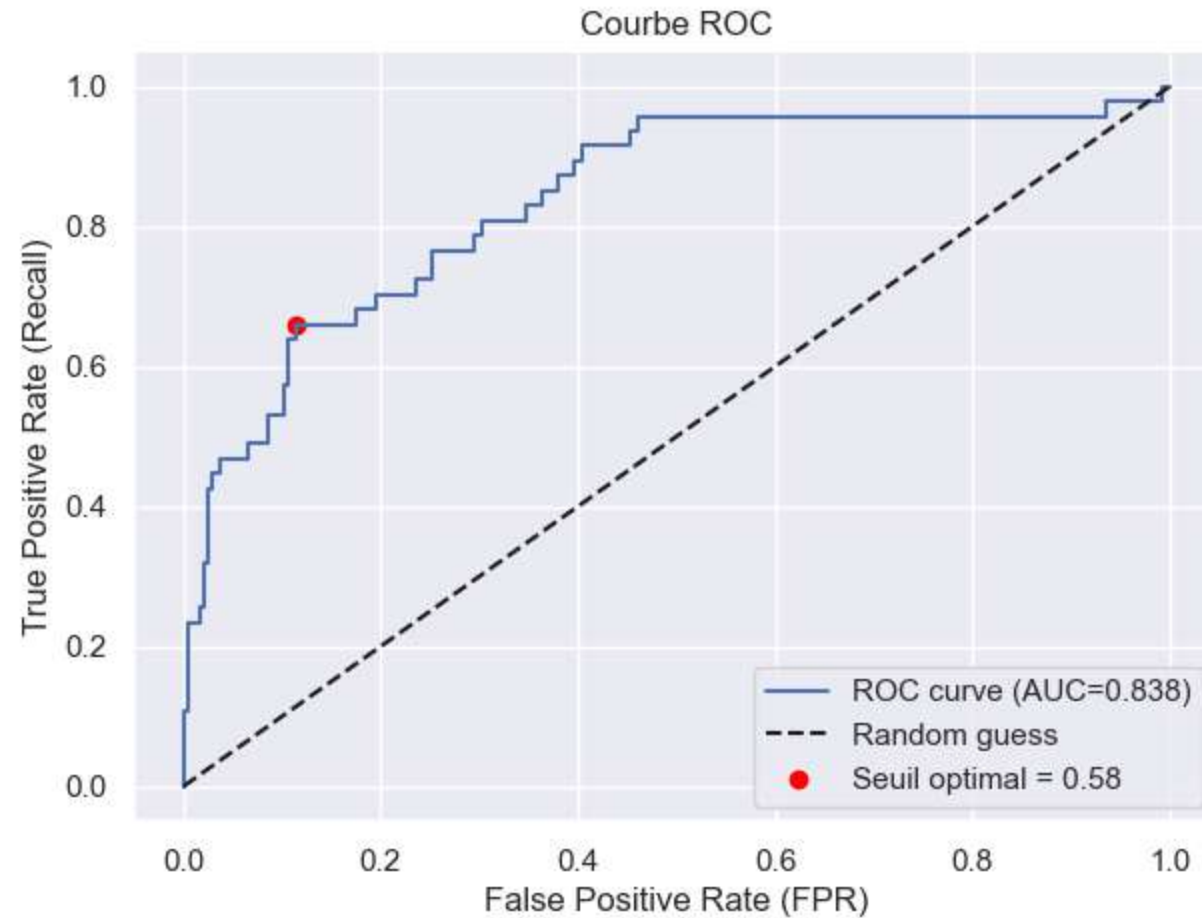
```
1 # Calcul du seuil optimal avec le Youden's J statistic
2 j_scores = tpr - fpr
3 j_ordered = sorted(zip(j_scores, thresholds), reverse=True)
4 best_threshold = j_ordered[0][1]
5
6 print("Seuil optimal:", best_threshold)
```

✓ 0.0s

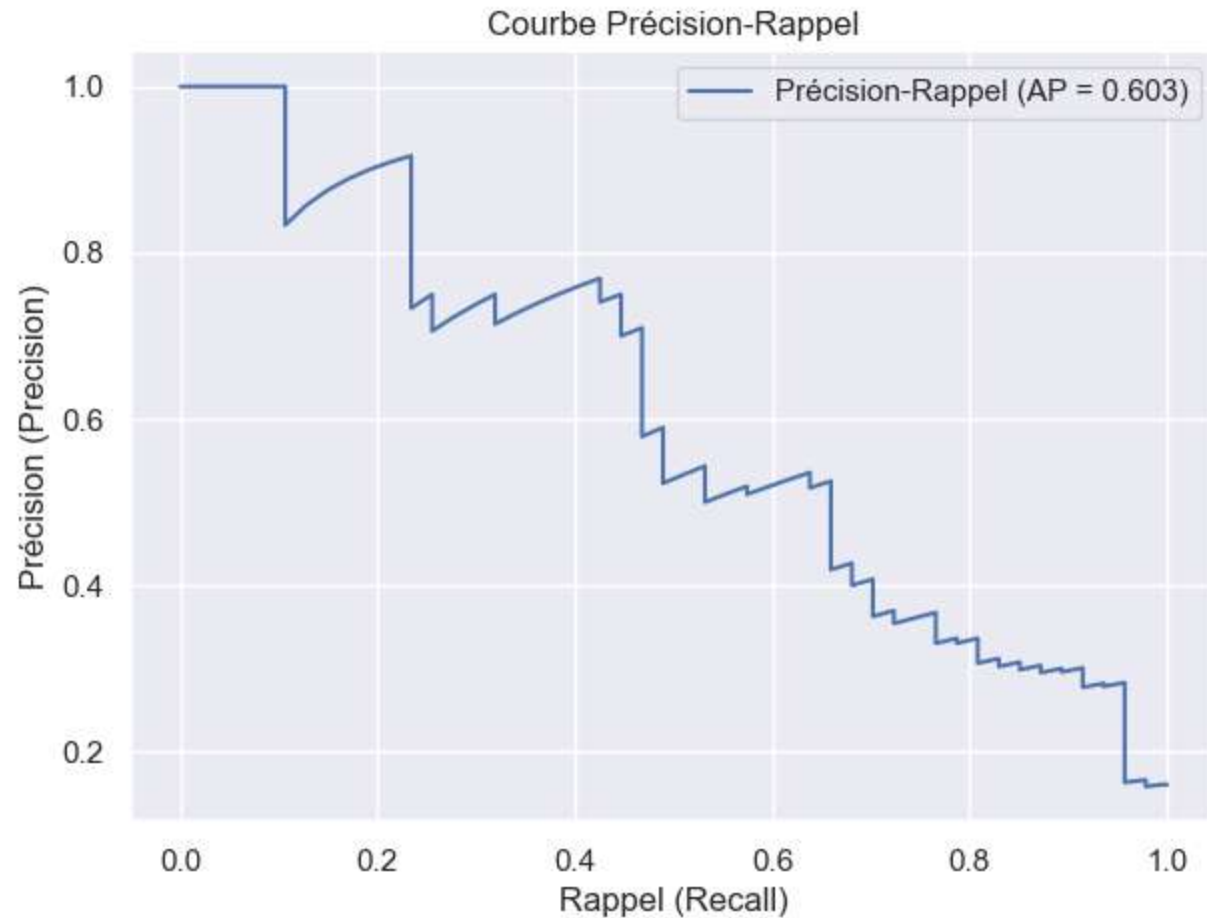
Seuil optimal: 0.5812455543107848

Seuil optimal

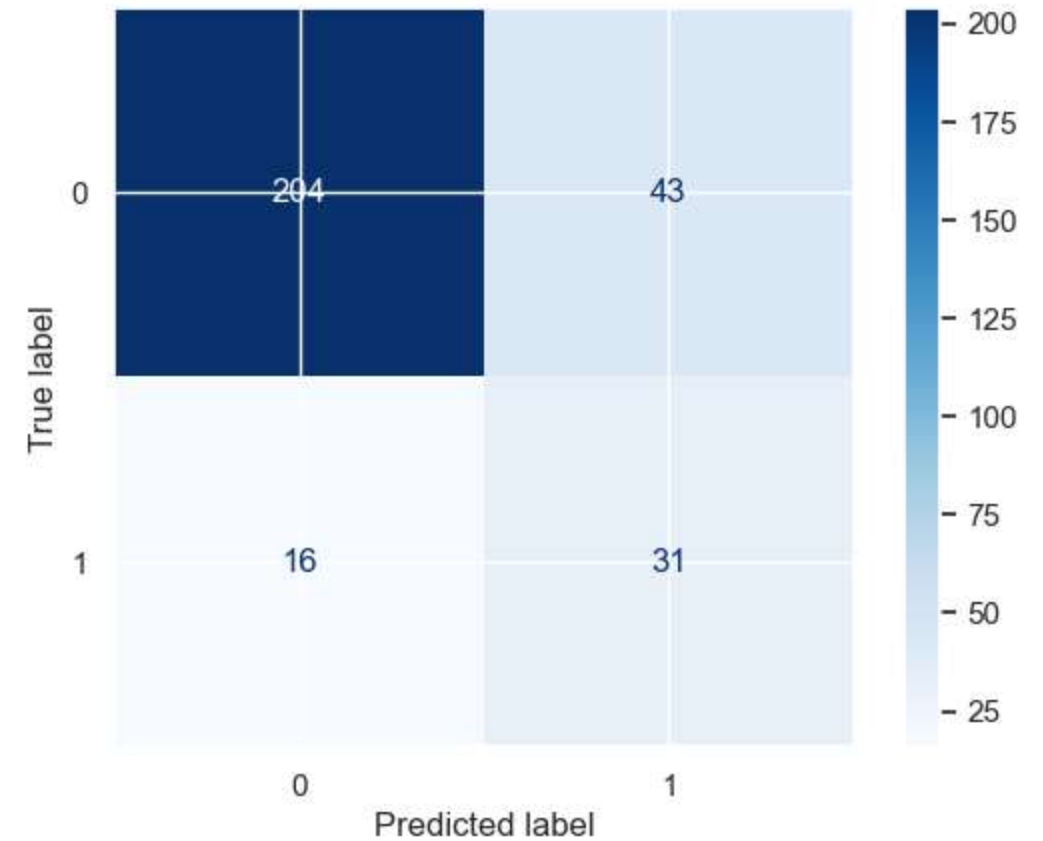
1. Calcul et affichage de la ROC



2. Graphique Précision-Rappel



3. Affichage de la Matrice de Confusion

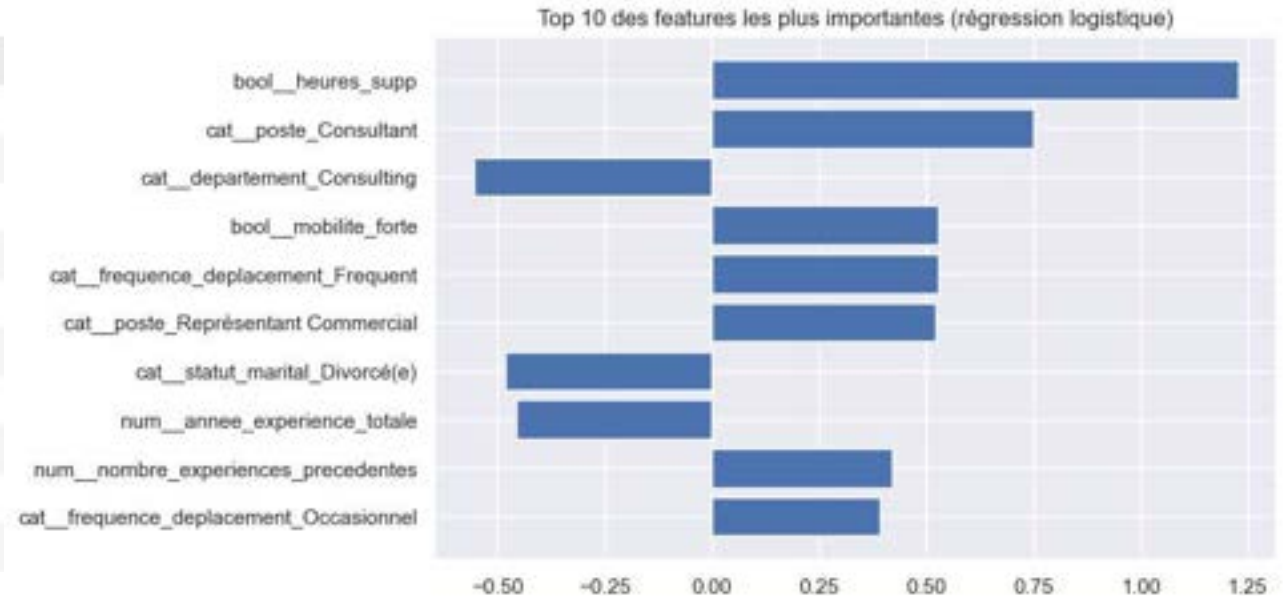


Explicabilité AI (XAI)

Features Importance & SHAP

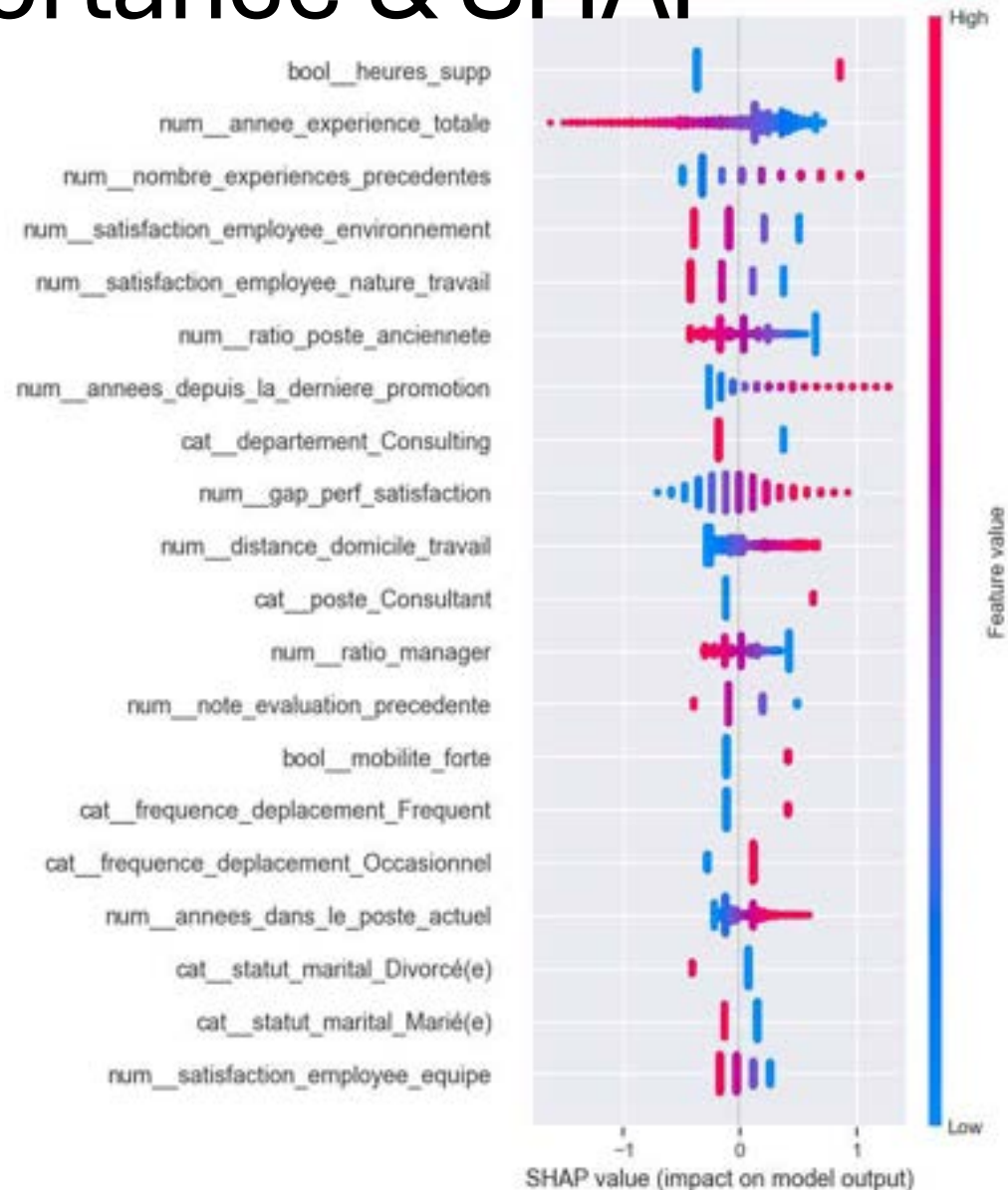
1. Features Importance

	feature	coefficient	abs_coeff
51	bool__heures_supp	1.229775	1.229775
33	cat__poste_Consultant	0.751179	0.751179
30	cat__departement_Consulting	-0.556361	0.556361
53	bool__mobilité_forte	0.529024	0.529024
45	cat__frequence_deplacement_Frequent	0.529024	0.529024
36	cat__poste_Représentant Commercial	0.521741	0.521741
28	cat__statut_marital_Divorcé(e)	-0.483375	0.483375
9	num__annee_experience_totale	-0.456212	0.456212
8	num__nombre_experiences_precedentes	0.420742	0.420742
46	cat__frequence_deplacement_Occasionnel	0.393715	0.393715



Features Importance & SHAP

2. A. SHAP global



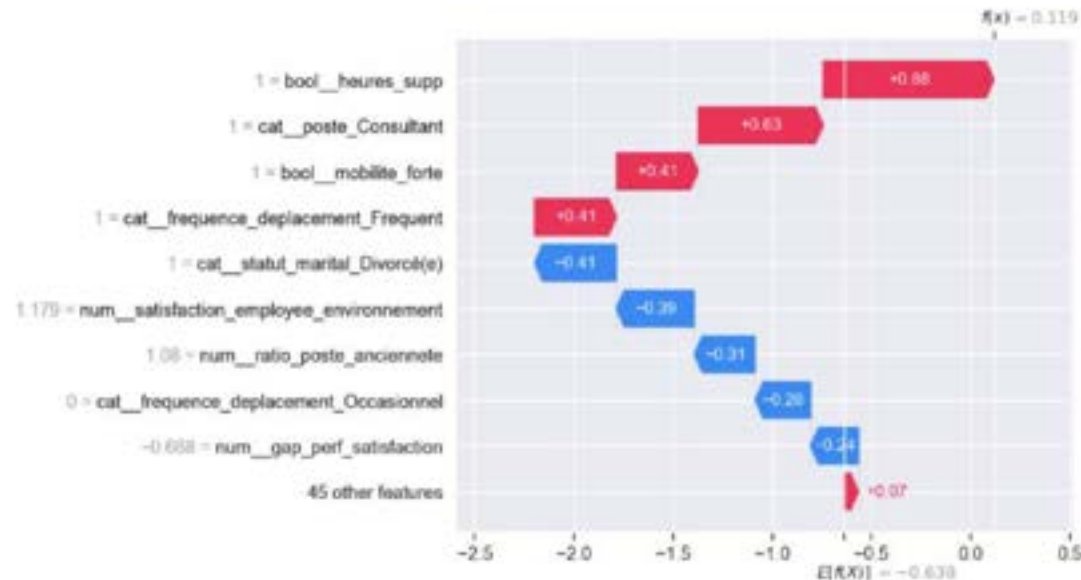
Features Importance & SHAP

2. B. SHAP local

I. Cas 1 : Sélection d'un individu parmi les True Positives - les "churners" bien détectés

```
1 import numpy as np
2
3 # indices des churn bien prédits
4 idx_churn = np.where((y_test == 1) & (y_pred == 1))[0]
5
6 i = idx_churn[0]
```

```
1 shap.initjs()
2
3 shap.waterfall_plot(
4     shap.Explainer(
5         values=shap_values[i],
6         base_values explainer.expected_value,
7         data=train_transformed[i],
8         feature_names=feature_names
9     )
10 )
```



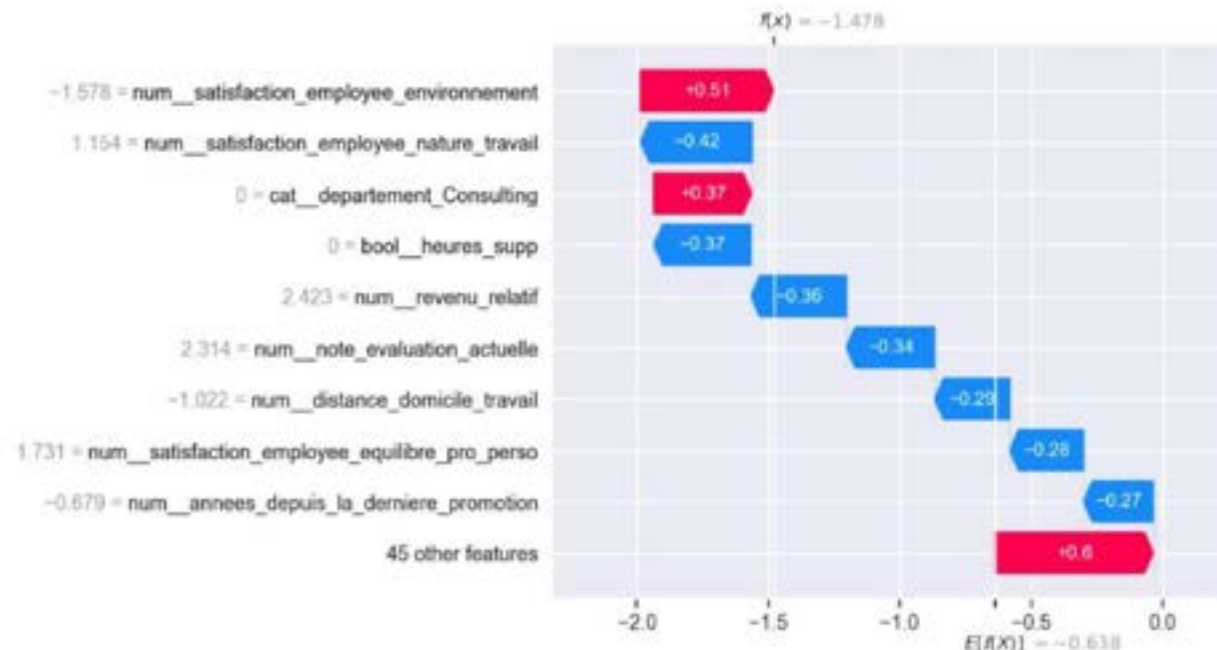
Features Importance & SHAP

ii. Cas 2 : Les Faux Négatifs que l'on veut minimiser le plus possible - les "churners" non détectés

2. B. SHAP local

```
1 idx_fn = np.where((y_test == 1) & (y_pred == 0))[0]
2
3 i = idx_fn[0]
✓ 0.0s

1 shap.initjs()
2
3 shap.waterfall_plot(
4     shap.Explanation(
5         values=shap_values[i],
6         base_values=explainer.expected_value,
7         data=train_transformed[i],
8         feature_names=feature_names
9     )
10 )
✓ 0.1s
```



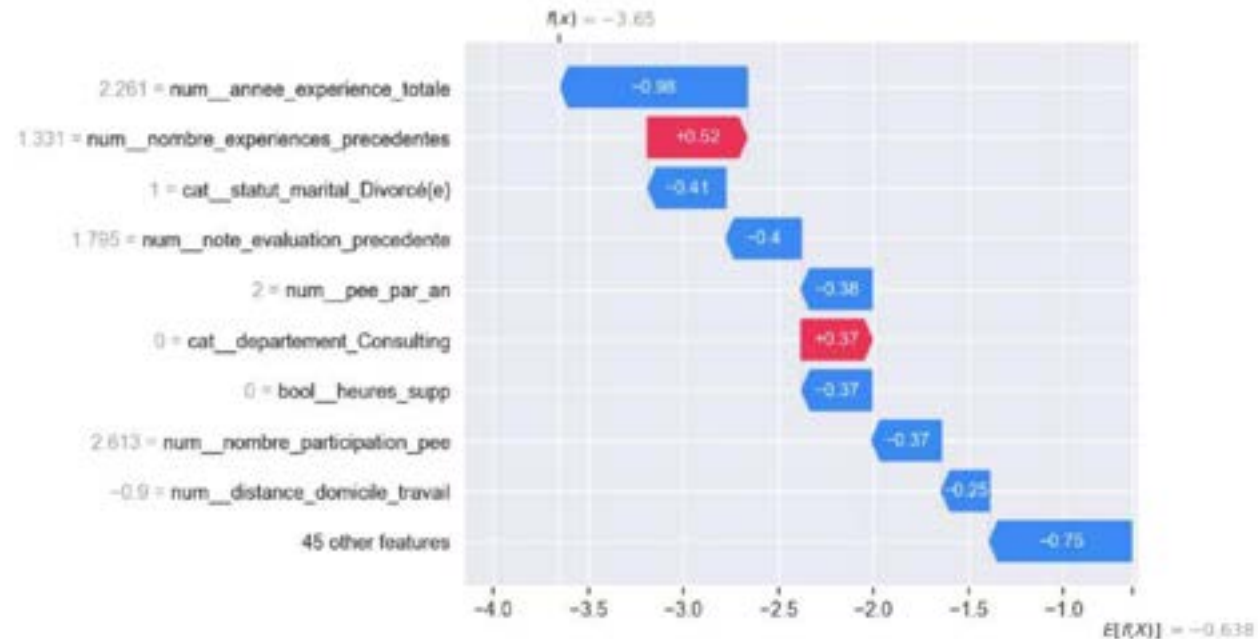
Features Importance & SHAP

2. B. SHAP local

iii. Cas 3 : False Positives - les fausses alertes

```
1 idx_fp = np.where((y_test == 0) & (y_pred == 1))[0]
2
3 i = idx_fp[0]
```

```
1 shap.initjs()
2
3 shap.waterfall_plot(
4     shap.Explanation(
5         values=shap_values[i],
6         base_values=explainer.expected_value,
7         data=X_train_transformed[i],
8         feature_names=feature_names
9     )
10 )
```



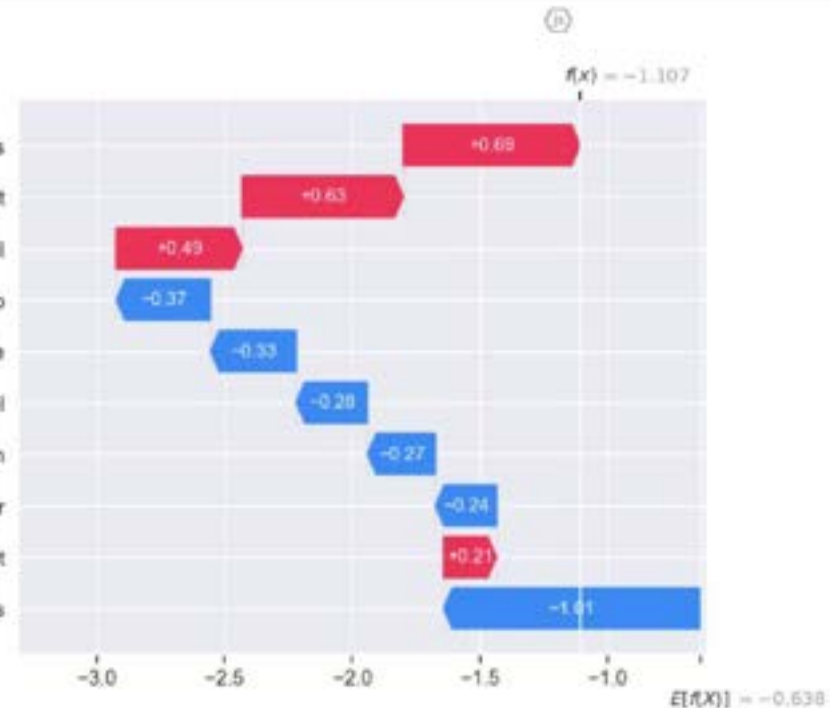
Features Importance & SHAP

iv. Cas 4 : Individu avec forte probabilité de churn

2. B. SHAP local

```
1 i = np.argmax(y_proba)
2 shap.initjs()
3
4 shap.waterfall_plot(
5     shap.Explanation(
6         values=shap_values[i],
7         base_values explainer.expected_value,
8         data=X_train_transformed[i],
9         feature_names=feature_names
10    )
11 )
```

1.733 = num_nombre_experiences_precedentes
1 = cat_poste_Consultant
1.791 = num_distance_domicile_travail
0 = bool_heures_supp
1.161 = num_ratio_poste_anciennete
0 = cat_frequentation_deplacement_Occasionnel
-0.679 = num_annees_depuis_la_derniere_promotion
1.167 = num_ratio_manager
-0.659 = num_satisfaction_employe_environnement
45 other features





Merci